COT 3100C: Discrete Structures

Lecture 02 — Logical Equivalence, Quantifiers, and SAT Solving

Course Details

01

Instructor

Luis Gonzalez, Ph.D.

02

Credits

4 Credits — 3 Lecture Hours + 2 Lab Hours per week

03

Prerequisite

COP 2800: Java Programming

Topics This Lecture

Logical Equivalence

Simplifying and transforming logical expressions using algebraic laws

Quantifiers

Predicate logic with universal and existential quantifiers

SAT Solving

Satisfiability, truth assignments, and the P vs. NP problem

Made with GAMMA

Lecture Agenda

This 75-minute lecture is structured into five timed segments, progressing from foundational prerequisites through to advanced topics in satisfiability and active learning exercises.

00m – 13m — 1

Prerequisites & Natural Language Ambiguity

2 — **13m – 28m**

Mathematical Operators & Truth Tables

28m – 48m — 3

Normal Forms (DNF/CNF) & Algebraic Laws

4 — **48m – 63m**

Worked Examples & Predicate Quantifiers

63m – 75m — 5

SAT, P vs. NP, & Active Learning

Made with GAMMA

Target Learning Outcomes

By the end of this lecture, students will be able to demonstrate competency across five core areas of discrete logic and formal reasoning.

1	**Logical Equivalence** Apply logical equivalences to simplify logical expressions
2	**Predicate Logic** Construct and interpret predicate logic with quantifiers
3	**Formal Proof** Prove logical equivalences using formal structural definitions
4	**Translation** Translate natural language into quantified predicate expressions
5	**Satisfiability** Determine formula satisfiability via truth assignments

Made with GAMMA

Motivation: The $1,000,000 "P vs. NP" Problem

The SAT Problem

The Boolean Satisfiability Problem (SAT) asks: given a logical formula, does there exist a truth assignment to its variables that makes the formula evaluate to **True**? This deceptively simple question sits at the heart of theoretical computer science.

The Central Question

Can SAT be solved **efficiently** — that is, in polynomial time? This is equivalent to asking whether **P = NP**, the most famous unsolved problem in all of computer science.

Real-World Impact

A proof that P = NP would **break modern cryptography** — RSA encryption, HTTPS, and virtually all secure communications rely on the assumption that certain problems cannot be solved efficiently.

$1M

Millennium Prize

Clay Mathematics Institute reward for solving P vs. NP

7

Millennium Problems

Total prize problems; P vs. NP is the only one with direct CS implications

P vs. NP has remained unsolved since it was formally posed in 1971. Most researchers believe P ≠ NP, but no proof exists.

Made with GAMMA

Prerequisite Review: Java Boolean Logic

This course builds directly on Boolean logic concepts from COP 2800 (Java Programming). The table below maps familiar Java operators to their formal mathematical counterparts used throughout this course.

Concept	Java Operator	Math Symbol	Alternate Notation
NOT (Negation)	`!`	¬	$\bar{P}$ or NOT P
AND (Conjunction)	`&&`	$\wedge$	P AND Q
OR (Disjunction)	`||`	$\vee$	P OR Q
Variable Domain	`boolean`	{T, F}	{true, false}

Continuity

Java's boolean type is a direct implementation of mathematical Boolean variables — the same rules apply in both contexts.

Notation Shift

This course uses formal mathematical notation (¬, $\wedge$, $\vee$) rather than programming syntax. Both represent identical logical operations.

Domain Constraint

Boolean variables are strictly binary — they take **only** the values True (T) or False (F). No other values are permitted.

Made with GAMMA

Misconception 1: "OR" in English vs. Math

The Everyday Ambiguity

Consider the statement: *"You may have cake, or you may have ice cream."*

In everyday English, this phrasing almost universally implies you must **choose only one**. The listener understands this as an **Exclusive-OR** — selecting both options would violate the social convention embedded in the sentence.

Why This Matters

Natural language is inherently ambiguous. The same word "or" can mean different things depending on context, tone, and cultural convention. Mathematics cannot tolerate this ambiguity — every operator must have a single, unambiguous definition.

The Mathematical Convention

In formal logic, the OR operator ($\vee$) is **always inclusive**. "Cake $\vee$ Ice Cream" is True whenever **at least one** of the operands is True — including the case where both are True.

P	Q	P $\vee$ Q
F	F	F
F	T	T
T	F	T
T	T	**T**

To express the exclusive sense, one must explicitly use **XOR ($\oplus$)**, which is False when both operands are True.

Made with GAMMA

Misconception 2: Logical Implication is NOT Causal

The Everyday Expectation

Consider: *"If pigs fly, then your account won't get hacked."*

In ordinary conversation, this statement sounds absurd — there is **no causal connection** between pigs flying and account security. A listener would reject it as meaningless or nonsensical.

The Mathematical Reality

Logical implication (→) is defined purely as a **truth function**. It makes no claim about causality, relevance, or meaningful connection between the hypothesis and conclusion. The operator is evaluated solely on the truth values of its operands.

> The implication P → Q is False **only** when P is True and Q is False. In all other cases — including when P is False — the implication is True.

Truth Table: Implication (P → Q)

P	Q	P → Q
F	F	**T** (vacuously true)
F	T	**T** (vacuously true)
T	F	**F** (only false case)
T	T	**T**

Evaluation of the Example

Since pigs do **not** fly, the hypothesis P is **False**. By the truth table, a false hypothesis yields a **True** implication — regardless of the conclusion. The statement is mathematically True.

Made with GAMMA

Misconception 3: The Power of False Hypotheses

This misconception has direct engineering consequences. Consider a formal system specification expressed as a conjunction of conditional rules:

$$\bigwedge_{i=1}^{12}(C_i \to A_i)$$

Variable Definitions

C_i — Sensor Condition

A Boolean variable representing whether sensor *i* is currently active (True) or inactive (False).

A_i — System Action

A Boolean variable representing whether the corresponding system action *i* is executed (True) or not (False).

The Dilemma

Suppose only **sensor 2** and **sensor 5** are active. The remaining 10 rules have a **False hypothesis** ($C_i = F$). Must those rules still be satisfied?

The Compliance Requirement

Yes — **all 12 rules** must hold simultaneously (that is the meaning of the conjunction ⋀). Inactive rules cannot be ignored; they must not falsify the specification.

The Resolution

By the definition of implication, a **False hypothesis must yield a True implication**. This is not a quirk — it is a deliberate design of the operator that makes system specifications well-defined even when most conditions are inactive.

> This is why "vacuous truth" is not a bug in logic — it is a feature that enables robust formal specification of real systems.

Made with GAMMA

Fundamental Terms: Propositions & Boolean Variables

Proposition

A **proposition** is a declarative sentence that is strictly either True (T) or False (F) — never both, never neither. This binary constraint is the foundation of classical propositional logic.

Valid Proposition

"The integer 7 is prime." — This is True, and it is unambiguously a proposition.

Not a Proposition

"Is 7 prime?" — A question has no truth value and is therefore not a proposition.

Propositional Variables

Symbols such as **P** and **Q** are used to represent arbitrary propositions. They serve as placeholders — the variable P might stand for any declarative statement, and its value is either T or F.

Boolean Variables

The term **Boolean** honors **George Boole** (1815–1864), the mathematician who first formalized this two-valued algebra in his 1854 work *An Investigation of the Laws of Thought*. Boolean variables are subject to a strict **domain constraint**: they can take on **only** the values T and F.

The binary domain {T, F} is what makes Boolean logic tractable — every formula over n variables has exactly 2^n possible truth assignments.

Made with GAMMA

Unary Operator: Negation (NOT)

Definition

Negation is a **unary operator** — it takes a single proposition as its operand and produces a new proposition with the **inverted truth value**. It is the simplest of all logical operators.

Notation

¬P

Primary formal notation used in this course and most logic textbooks.

P̄

Overbar notation, common in digital circuit design and electrical engineering.

NOT P

Verbose English notation, used for clarity in introductory contexts.

Function

Negation **inverts** the truth value of its operand: a True proposition becomes False, and a False proposition becomes True. It is the logical analog of the arithmetic unary minus.

Truth Table: Negation (¬P)

P	¬P
T	**F**
F	**T**

Key Properties

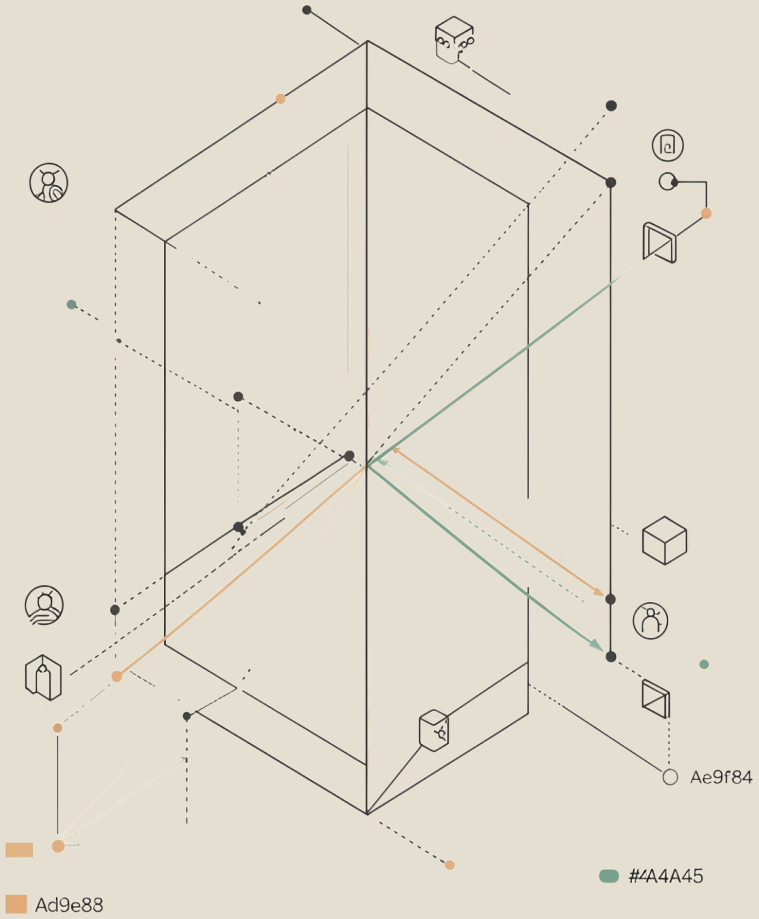
Double Negation

¬(¬P) ≡ P — Negating twice returns the original proposition. This is a fundamental algebraic law.

Unary Scope

¬ applies to exactly one operand. To negate a compound expression, parentheses must be used: ¬(P ∧ Q).

Made with GAMMA



Propositional Logic: Binary Operators & Logical Relationships

An introduction to the fundamental building blocks of logical reasoning for computer science.

Made with GAMMA

Binary Conjunction: AND

The **conjunction** is the logical combination of two propositions, written $P \wedge Q$ (or P AND Q).

The AND operator evaluates to True **only** when both P and Q are True. With 2 variables, the truth table has exactly 4 rows — one for each possible assignment.

P	Q	P ∧ Q
T	T	**T**
T	F	F
F	T	F
F	F	F

Made with GAMMA

Binary Disjunction: OR

P	Q	P ∨ Q
T	T	**T**
T	F	**T**
F	T	**T**
F	F	F

Inclusive-OR

The **disjunction** is the mathematical inclusive-OR combination, written $P \vee Q$ (or P OR Q).

The OR operator is True if P, Q, or **both** are True. It evaluates to False **only** when both inputs are False — the single row at the bottom of the table.

Made with GAMMA

Binary Exclusion: XOR

Definition

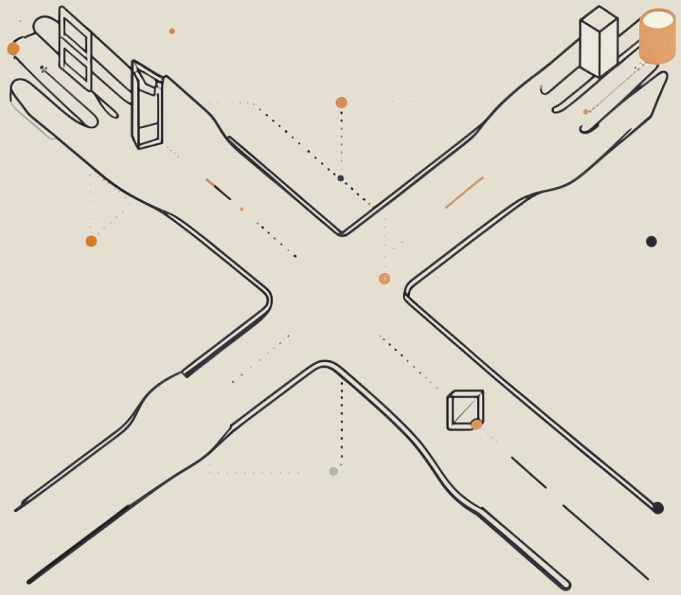
Exclusive-OR restricts the choice to exactly one. Written $P \oplus Q$ (or P XOR Q).

True When

Evaluates to True if **exactly one** of P or Q is True — but not both.

Double-True Case

Evaluates to **False** when both P and Q are True — the key difference from OR.



Made with GAMMA

The Biconditional: IFF (If and Only If)

Asserts Identical Truth Values

The **biconditional** is written $P \leftrightarrow Q$ (or P IFF Q). It is True when both propositions share the same truth value — both True, or both False.

Real Example:

> $x^2 - 4 = 0 \leftrightarrow |x| = 2$ is always True

Both sides are satisfied by exactly the same values of x, making the biconditional a tautology in this case.

P	Q	P ↔ Q
T	T	**T**
T	F	F
F	T	F
F	F	**T**

Made with GAMMA

The Conditional: IMPLIES

Structure of an Implication

Written $P \to Q$ (or P IMPLIES Q), where P is the **hypothesis** (if-part) and Q is the **conclusion** (then-part).

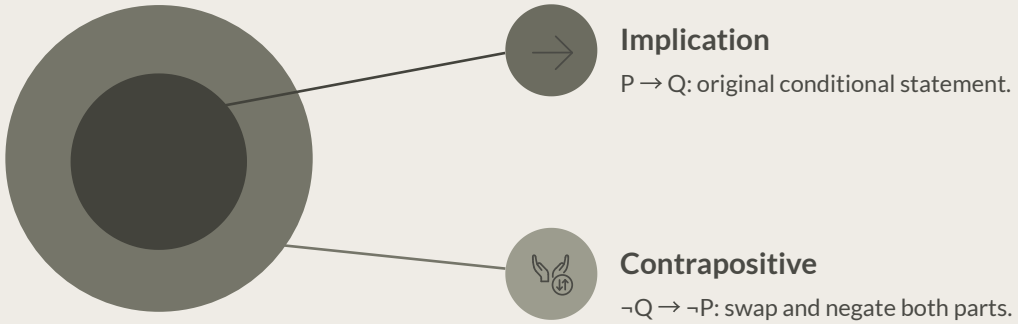
The conditional evaluates to False **only** when P is True and Q is False. In all other cases it is True.

Summary: True exactly when the if-part is false or the then-part is true.

P	Q	P → Q
T	T	**T**
T	F	F
F	T	**T**
F	F	**T**

Made with GAMMA

Implication Relationships: The Contrapositive



Swaps and Negates

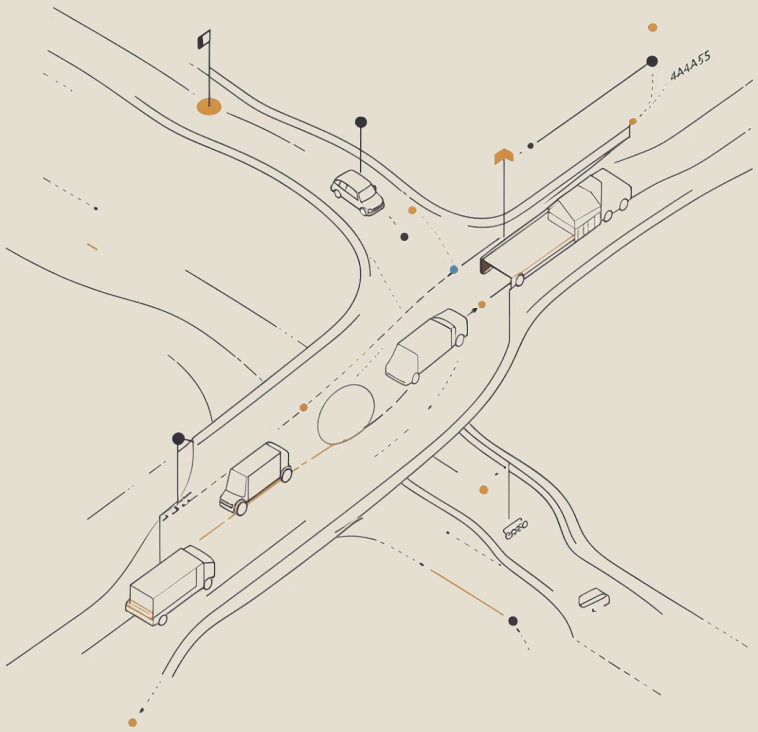
The **contrapositive** of $P \to Q$ is $\neg Q \to \neg P$. Both the hypothesis and conclusion are swapped *and* negated.

Example: Starting from "If I am hungry, then I am not grumpy," the contrapositive is:

> "If I am grumpy, then I am not hungry"

Logical Law: An implication is always equivalent to its contrapositive — they share matching truth value columns in every row.

Made with GAMMA



Implication Relationships: The Converse

The Converse

Swaps variables **without** negating: $Q \to P$. The hypothesis and conclusion simply trade places.

Example

From "If I am hungry, then I am not grumpy," the converse is: *"If I am grumpy, then I am hungry."*

Logical Law

An implication is **not** equivalent to its converse. Their truth columns differ in the second and third rows.

Made with GAMMA

Connecting Implication, Converse, and IFF

The Key Identity

$$(P \to Q) \land (Q \to P) \equiv P \leftrightarrow Q$$

Combining an implication with its converse using AND produces exactly the biconditional.

> "If I'm grumpy then I'm hungry, AND vice versa" ≡ "I am grumpy IFF I am hungry"

Truth Verification: The AND of the two implication columns matches the IFF column in every row.

P	Q	P→Q	Q→P	P↔Q
T	T	T	T	**T**
T	F	F	T	**F**
F	T	T	F	**F**
F	F	T	T	**T**

Made with GAMMA

Formula Classifications: Equivalence, Validity, & Satisfiability

Valid Formula (Tautology)

Always True under **every** assignment. Example: $P \vee \neg P$ is always True regardless of the value of P.

Equivalence

$F \equiv G$ holds precisely when $F \leftrightarrow G$ is valid — their truth tables match in every row.

Satisfiable Formula

Sometimes True under **at least one** assignment. P is satisfiable IFF $\neg P$ is **not** valid.



Made with GAMMA

Structural Units: Literals and AND-clauses

Literal

A variable (A) or its negation (Ā). The atomic building block of all propositional logic formulas.

AND-clause

A conjunction of literals with **distinct** variables. No variable may appear more than once within a single clause.

Valid vs. Invalid Examples

Status	Expression	Reason
✓ Valid	A ∧ B ∧ C	All variables distinct
⍰ Invalid	A ∧ B ∧ B ∧ C	B appears twice

As computer scientists, we often need to transform messy, arbitrary logical formulas into standardized formats called **normal forms**. Defining their structural components is the essential first step. A literal is simply a single propositional variable, like A, or its negation, like Ā. An AND-clause is a conjunction of literals, but with a critical constraint: the literals must involve distinct variables.

Made with GAMMA

Disjunctive Normal Form (DNF)

DEFINITION

DNF: An OR of AND-clauses

Also known as a **sum-of-products** representation — a term borrowed from circuit design, where AND acts as a product and OR acts as a sum.

Formula Structure

$\text{Clause}_1 \vee \text{Clause}_2 \vee \ldots \vee \text{Clause}_k$

All conjunctions are nested inside individual clauses. Those clauses are joined at the top level **exclusively** by disjunctions.

Canonical Example

$(A \wedge B) \vee (A \wedge C)$

Key Structural Rules

- Each clause is an AND-clause (literals with distinct variables)
- Clauses are joined only by OR at the top level
- No OR operators may appear inside a clause
- One or more clauses may be present

Made with GAMMA

Conjunctive Normal Form (CNF)

DEFINITION

CNF: An AND of OR-clauses

Also known as a **product-of-sums** representation. Each OR-clause is a disjunction of literals from distinct variables.

Canonical Example

$(A \vee B \vee C) \wedge (\bar{A} \vee \bar{B} \vee \bar{C})$

CNFs are highly structured and serve as the standard input format for practical **SAT-solver** software.

DNF vs. CNF at a Glance

Property	DNF	CNF
Top-level operator	OR ($\vee$)	AND ($\wedge$)
Clause type	AND-clause	OR-clause
Circuit alias	Sum-of-products	Product-of-sums
Primary use	Logic synthesis	SAT solving

Made with GAMMA

The Normal Form Theorem

THEOREM 3.4.1

> Every propositional formula, no matter how complex or nested, is logically equivalent to both a full DNF and a full CNF.

What is a "Full" Normal Form?

A normal form is **full** when every single clause in the expression contains exactly one literal for every variable present in the entire formula. No variable may be absent from any clause.

DNF Representation

Every formula is equivalent to a full DNF — an OR of AND-clauses, each containing a literal for every variable.

CNF Representation

Every formula is equivalently expressible as a full CNF — an AND of OR-clauses, each containing a literal for every variable.

Why This Matters

1 Standardization

Any logical statement can be rewritten in a canonical, predictable structure regardless of its original form.

2 Compiler Optimization

Normal forms enable systematic simplification of boolean expressions in compiler backends.

3 Hardware Verification

Circuit designers rely on CNF/DNF to verify the correctness of logic gate configurations.

Made with GAMMA

Algebraic Axioms: Conjunction Laws

While truth tables are reliable, they grow **exponentially** with the number of variables. Algebraic axioms provide an alternative path to proving logical equivalence. Many of these laws mirror the arithmetic properties learned in grade school.

Commutativity

$A \wedge B \equiv B \wedge A$

Order of operands does not affect the result.

Associativity

$(A \wedge B) \wedge C \equiv A \wedge (B \wedge C)$

Parentheses may be freely dropped or rearranged.

Identity

$T \wedge A \equiv A$

True is the identity element for conjunction.

Zero

$F \wedge A \equiv F$

False is the zero (annihilator) element for conjunction.

Distributivity (AND over OR)

$A \wedge (B \vee C) \equiv (A \wedge B) \vee (A \wedge C)$

Mirrors multiplication distributing over addition.

Made with GAMMA

Algebraic Axioms: Distributivity

The Two Distributivity Laws

Law 1 — AND over OR

$A \wedge (B \vee C) \equiv (A \wedge B) \vee (A \wedge C)$

Behaves exactly like multiplication distributing over addition. Has a direct arithmetic parallel.

Law 2 — OR over AND

$A \vee (B \wedge C) \equiv (A \vee B) \wedge (A \vee C)$

No numerical parallel exists. In regular algebra, $x + (y \times z) \neq (x + y) \times (x + z)$. In boolean logic, this equivalence is perfectly valid.

Crucial Difference from Arithmetic

Propositional logic has **two** distributivity laws. The first mirrors arithmetic. The second is unique to boolean algebra and has no counterpart in standard mathematics.

⚠ OR distributing over AND is a frequent source of errors when students first write algebraic logic proofs. There is no arithmetic analogue — do not assume one exists.

Mathematical Significance

This property is what makes boolean algebra a **complemented distributive lattice** — a structure richer than ordinary arithmetic rings, enabling the full duality between AND and OR that underlies all of propositional logic.

Made with GAMMA

Algebraic Axioms: Negation, Idempotence & Validity

Beyond commutativity and distributivity, we must establish how variables interact with themselves and their negations. These laws govern the self-referential behavior of propositional variables.

Idempotence

$A \wedge A \equiv A$

Repeating a variable with AND does not change its value. Redundant conjuncts can be eliminated.

Contradiction

$A \wedge \bar{A} \equiv F$

A variable and its negation cannot both be true simultaneously. The conjunction always evaluates to False.

Double Negation

$\neg(\neg A) \equiv A$

Two negations cancel out, returning the original variable. Negation is its own inverse.

Validity (OR)

$A \vee \bar{A} \equiv T$

Either a statement is true or its negation is true. This disjunction always evaluates to True.

Made with GAMMA

De Morgan's Laws

CORE PRINCIPLE

Distributing Negation Across a Clause

De Morgan's Laws are among the most frequently applied tools in logical simplification and programming. They dictate how to distribute a negation across an entire parenthesized clause.

The mechanics are simple but critical: distribute the negation to each literal inside, then **flip the operator**. AND becomes OR; OR becomes AND.

De Morgan's Laws are easily verified via truth table evaluation for any two-variable case.

De Morgan for AND

$\neg(A \wedge B) \equiv \bar{A} \vee \bar{B}$

The negation of a conjunction is the disjunction of the negations.

De Morgan for OR

$\neg(A \vee B) \equiv \bar{A} \wedge \bar{B}$

The negation of a disjunction is the conjunction of the negations.

The Two-Step Rule

1. Distribute ¬ to each literal inside the parentheses
2. Flip the operator: $\wedge$ ↔ $\vee$

Made with GAMMA

Theorem 3.4.2: Equivalence to Canonical Form

THEOREM 3.4.2

> Any formula can be proved equivalent to its canonical form using only the propositional algebraic axioms.

What is Canonical Form?

A canonical form is a **unique, standardized representation** of a formula's truth table. It is sorted and stripped of all duplicate clauses, yielding a single unambiguous expression for any given truth function.

Canonical DNF

A sorted, full DNF with no duplicate clauses. Every clause contains exactly one literal per variable; clauses are ordered consistently.

Algebraic Tool

The conversion uses only the propositional axioms established in the preceding slides — no truth tables required.

Why This is a Major Milestone

→ Axioms Are Complete

The algebraic axioms are not a random collection of rules. They are powerful enough to transform *any* logical formula into canonical form.

→ Uniqueness Guaranteed

The canonical form is unique for any given truth table. If two formulas are logically equivalent, they reduce to the **exact same** canonical form.

→ Equivalence Checking

To test whether two formulas are equivalent, reduce both to canonical form and compare. Equality of canonical forms is a **decision procedure** for propositional equivalence.

Made with GAMMA

Theorem 3.4.3: Completeness of Axioms

PROPOSITIONAL LOGIC · SLIDE 29

Theorem 3.4.3 (Completeness)

A formula is proven equivalent **if and only if** it can be proven via the algebraic axioms of propositional logic. This is not merely a convenience — it is a foundational guarantee about the power of the axiomatic system.

Guaranteed Proof

If an equivalence exists, an algebraic proof exists. The axiomatic path is always open.

Theoretical Impact

Eliminates the mandatory need for truth tables. Algebraic laws are sufficient to establish any logical equivalence.

The Catch

Algebraic proofs can match truth table complexity in the worst case — completeness guarantees existence, not brevity.

> Theorem 3.4.3 establishes the **"Completeness"** of propositional logic. This theorem reassures us that our set of algebraic axioms is complete: if two formulas are logically equivalent, it is mathematically guaranteed that we can prove their equivalence using only these algebraic laws.

This means we never have to rely on giant, brute-force truth tables to prove a theorem; the algebraic path is always open and guaranteed to succeed.

Completeness is a meta-theorem: it makes a claim *about* the proof system itself, not about any particular formula.

Made with GAMMA

The Bottleneck: Truth Table Exponential Growth

COMPUTATIONAL COMPLEXITY · SLIDE 30

The number of rows in a truth table grows as **2n**, where **n** is the number of propositional variables. This exponential scaling makes brute-force verification rapidly impractical.

8

3 Variables

Rows required for complete verification

1K

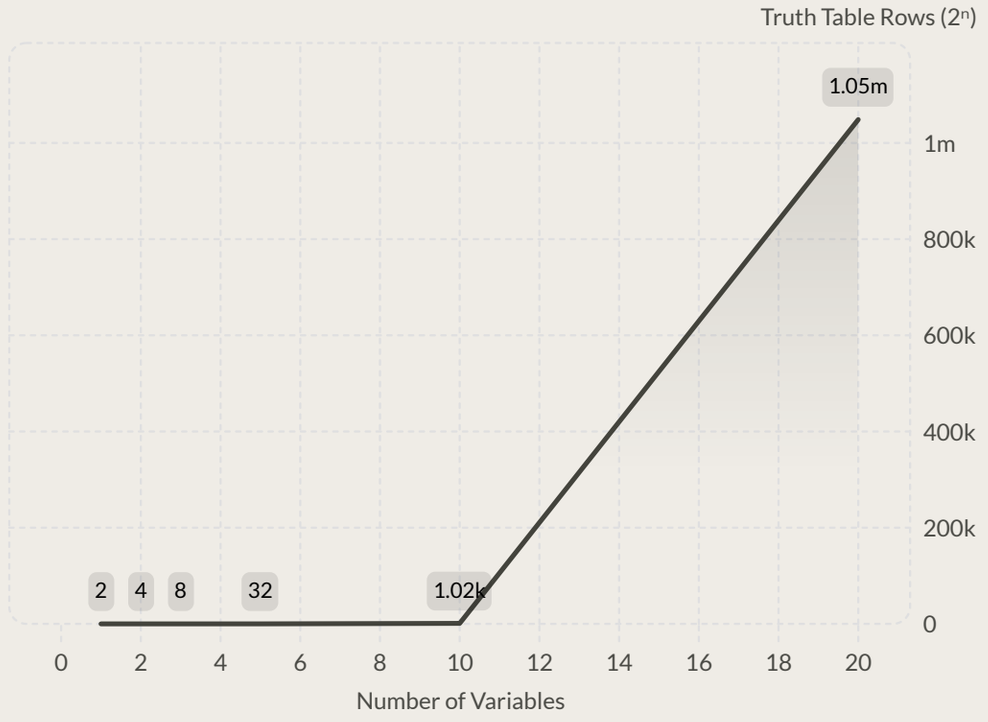
10 Variables

Over 1,024 rows to check exhaustively

1B+

30 Variables

Over one billion rows — computationally intractable



For just thirty variables — a very small number for a modern software application or circuit design — the truth table requires over a billion rows. Checking this by hand, or even with a computer, quickly becomes impractical. This is precisely why algebraic proof methods are indispensable.

Made with GAMMA

Worked Case Step 1: Simplifying Java Code

CASE STUDY · SLIDE 31

Original Java Conditional

```
if ( x > 0 || (x <= 0 && y > 100) )
```

This conditional appears in a legacy codebase. To analyze it rigorously, we translate it into formal propositional logic by defining atomic propositions for each sub-expression.

Proposition A

Let **A** represent the assertion `x > 0`

Proposition B

Let **B** represent the assertion `y > 100`

Negation Ā

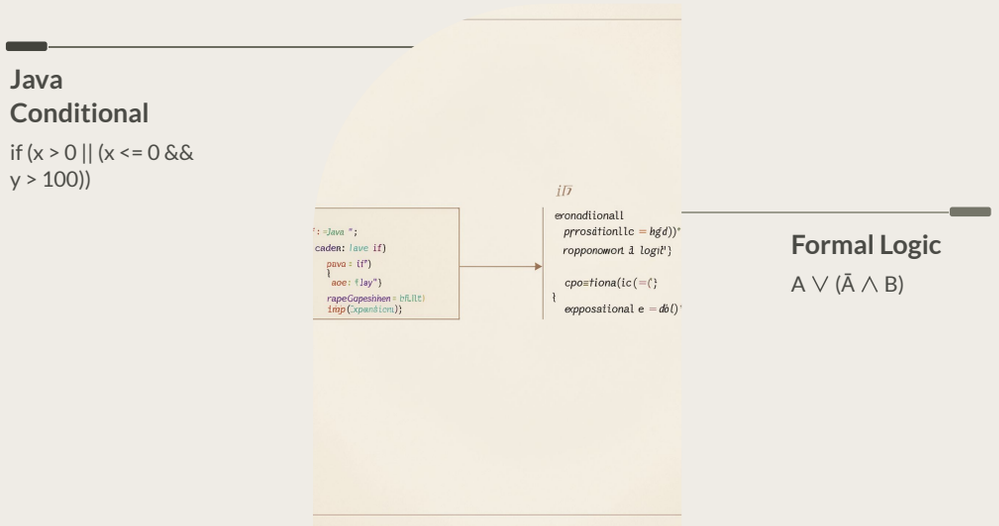
`x <= 0` is the negation of A, written **Ā**

Formal Translation

Since `x` must be either greater than zero or less-than-or-equal to zero, `x <= 0` is simply the negation of A. Our formal expression is therefore:

> **A ∨ (Ā ∧ B)**

This reads: *"x is greater than zero, OR (x is not greater than zero AND y is greater than 100)."*



Made with GAMMA

Worked Case Step 2: The Truth Table Proof

CASE STUDY · SLIDE 32

Evaluating: A ∨ (Ā ∧ B) vs. A ∨ B

A	B	Ā	Ā ∧ B	A ∨ (Ā ∧ B)	A ∨ B
T	T	F	F	**T**	**T**
T	F	F	F	**T**	**T**
F	T	T	T	**T**	**T**
F	F	T	F	**F**	**F**

Equivalence Proven

The final two columns — **A ∨ (Ā ∧ B)** and **A ∨ B** — match exactly across all four rows: **True, True, True, False**. This proves logical equivalence.

The Java conditional can be safely refactored to: `if (x > 0 || y > 100)`

The intermediate column for **Ā ∧ B** displays the conjunction values. The disjunction of A with that column yields the final output, which is identical to the simpler expression A ∨ B in every row.

Made with GAMMA

Worked Case Step 3: Proof by Cases

CASE STUDY · SLIDE 33

While truth tables are straightforward, the same equivalence can be proven using **case analysis** — splitting the universe based on the truth value of a key variable.

Case 1: A is True

Anything ORed with True is automatically True.

- Complex: $\mathbf{T \vee (F \wedge B) \equiv T \vee F \equiv T}$
- Simple: $\mathbf{T \vee B \equiv T}$
- Both expressions evaluate to **True** ✓

Case 2: A is False

A OR B simplifies directly to B when A is False.

- Complex: $\mathbf{F \vee (T \wedge B) \equiv T \wedge B \equiv B}$
- Simple: $\mathbf{F \vee B \equiv B}$
- Both expressions evaluate to **B** ✓

> In both cases, the outputs match perfectly. This constitutes a complete proof by cases that $\mathbf{A \vee (\bar{A} \wedge B) \equiv A \vee B}$, without constructing a single truth table row.

Made with GAMMA

Worked Case Step 4: Deriving Full DNF

CASE STUDY · SLIDE 34

The DNF Construction Rule

Extract the rows from the truth table where the output is **True**. For each such row, write an AND-clause containing a literal for each variable — the variable itself if True, its negation if False.

Row	A	B	Output	Clause
1	T	T	**T**	$A \wedge B$
2	T	F	**T**	$A \wedge \bar{B}$
3	F	T	**T**	$\bar{A} \wedge B$
4	F	F	F	—

Resulting Full DNF

> $$(A \wedge B) \vee (A \wedge \bar{B}) \vee (\bar{A} \wedge B)$$

The three True rows yield three AND-clauses. Joining them with ORs produces the **Full Disjunctive Normal Form**. Every satisfying assignment of the formula corresponds to exactly one clause in the DNF.

01

Identify True Rows

Rows 1, 2, and 3 have output True.

02

Write AND-Clauses

One clause per True row, using literals matching the row's assignment.

03

Join with OR

Disjoin all clauses to form the complete DNF expression.

Made with GAMMA

Worked Case Step 5: Deriving Full CNF

CASE STUDY · SLIDE 35

The CNF Construction Rule

To construct a **Conjunctive Normal Form**, perform the dual operation: extract the rows where the output is **False**. For each such row, write an OR-clause designed to evaluate to False only for that specific assignment.

Row	A	B	Output	Clause
1	T	T	T	—
2	T	F	T	—
3	F	T	T	—
4	F	F	**F**	A ∨ B

Resulting Full CNF

> **(A ∨ B)**

There is only one False row: row 4, where both A and B are False. The OR-clause **A ∨ B** evaluates to False only when both A and B are False — precisely matching row 4's assignment.

Single False Row

Only one OR-clause is needed. The Full CNF is simply **(A ∨ B)**.

Multiple False Rows

If there were multiple False rows, construct one OR-clause per row and join them all with ANDs.

DNF and CNF are dual constructions: DNF targets True rows with AND-clauses; CNF targets False rows with OR-clauses.

Made with GAMMA

Worked Case Step 6: Algebraic Reduction (Part I)

CASE STUDY · SLIDE 36

Target Formula

$$\neg((A \wedge B) \vee (A \wedge C))$$

We simplify the negation of **(A AND B) OR (A AND C)** using De Morgan's Laws to push negations inward until they apply only directly to variables.

01

De Morgan for OR

Distribute the outermost negation. The top-level OR becomes AND:

$$\neg(A \wedge B) \wedge \neg(A \wedge C)$$

02

De Morgan for AND

Distribute negations into both inner clauses. Each AND becomes OR:

$$(\bar{A} \vee \bar{B}) \wedge (\bar{A} \vee \bar{C})$$

State After Part I

After applying De Morgan's Laws twice, the NOTs now apply **only directly to variables** — not to compound sub-expressions. This is a prerequisite for converting to normal form.

Step	Expression	Law Applied
Start	$\neg((A \wedge B) \vee (A \wedge C))$	—
1	$\neg(A \wedge B) \wedge \neg(A \wedge C)$	De Morgan (OR)
2	$(\bar{A} \vee \bar{B}) \wedge (\bar{A} \vee \bar{C})$	De Morgan (AND)

> NOTs now only apply directly to variables. The formula is in **Negation Normal Form (NNF)** — ready for further reduction.

Made with GAMMA

Worked Case Step 7: Algebraic Reduction (Part II)

CASE STUDY · SLIDE 37

Continuing from **$(\bar{A} \vee \bar{B}) \wedge (\bar{A} \vee \bar{C})$**, we apply distributivity and idempotence to reach a clean DNF.

Step	Expression	Law Applied
2 (carry-in)	$(\bar{A} \vee \bar{B}) \wedge (\bar{A} \vee \bar{C})$	De Morgan (from Part I)
3	$((\bar{A} \vee \bar{B}) \wedge \bar{A}) \vee ((\bar{A} \vee \bar{B}) \wedge \bar{C})$	Distributivity: AND over OR
4	$(\bar{A} \wedge \bar{A}) \vee (\bar{B} \wedge \bar{A}) \vee (\bar{A} \wedge \bar{C}) \vee (\bar{B} \wedge \bar{C})$	Distributivity (expand further)
5	$\bar{A} \vee (\bar{B} \wedge \bar{A}) \vee (\bar{A} \wedge \bar{C}) \vee (\bar{B} \wedge \bar{C})$	Idempotence: $\bar{A} \wedge \bar{A} \equiv \bar{A}$

Key Moves Explained

1 Distribute AND over OR

The left clause **$(\bar{A} \vee \bar{B})$** is distributed over the right clause **$(\bar{A} \vee \bar{C})$**, expanding into four individual AND-clauses.

2 Apply Idempotence

The clause $\bar{A} \wedge \bar{A}$ contains a duplicate literal. By the Idempotence Law, $X \wedge X \equiv X$, so this simplifies directly to $\bar{A}$.

Final DNF Result

> **$\bar{A} \vee (\bar{B} \wedge \bar{A}) \vee (\bar{A} \wedge \bar{C}) \vee (\bar{B} \wedge \bar{C})$**

The formula is now in **Disjunctive Normal Form**: a disjunction of conjunctions of literals. Each AND-clause represents one satisfying pattern. The algebraic reduction is complete — no truth table was required.

Made with GAMMA

Worked Case Step 8: Quantifier Ordering & English Ambiguity

PREDICATE LOGIC · SLIDE 38

The Ambiguous Sentence

> **"Every American has a dream."**

Natural language is notoriously ambiguous when combining universal and existential quantifiers. The same English sentence admits two fundamentally different logical interpretations depending on quantifier order.

Variables

- **A** — the set of Americans
- **D** — the set of dreams
- **H(a, d)** — predicate: person *a* holds dream *d*

Interpretation 1: Shared Dream

> $\exists d \in D\ \forall a \in A: H(a, d)$

"There exists a single dream d such that every American a holds that dream." The existential quantifier comes first — one dream is shared by all.

Interpretation 2: Personal Dreams

> $\forall a \in A\ \exists d \in D: H(a, d)$

"For every American a, there exists some dream d that a holds." The universal quantifier comes first — each person may have their own distinct dream.

⚠ Swapping the order of quantifiers ($\exists\ \forall$ vs. $\forall\ \exists$) completely changes the fundamental meaning of the expression. Interpretation 1 is strictly stronger than Interpretation 2.

Interpretation 1 implies Interpretation 2, but not vice versa. This asymmetry is a core principle of predicate logic: **quantifier order is not commutative** when mixing $\exists$ and $\forall$.

Made with GAMMA

Software and Hardware Optimization

SLIDE 39

Why do computer science and engineering majors need to master logical simplification? In software, simplifying complex conditionals makes code highly readable and faster to execute because the CPU performs fewer operations. In hardware design, digital circuits are represented directly by logical formulas — minimizing these formulas means building the same processor with fewer physical logic gates.

Software Readability

Simplified code is easier to maintain. Fewer conditional branches reduce cognitive load and debugging time.

Performance

Fewer operations mean faster runtimes. The CPU executes fewer instructions per logical decision.

Hardware Silicon

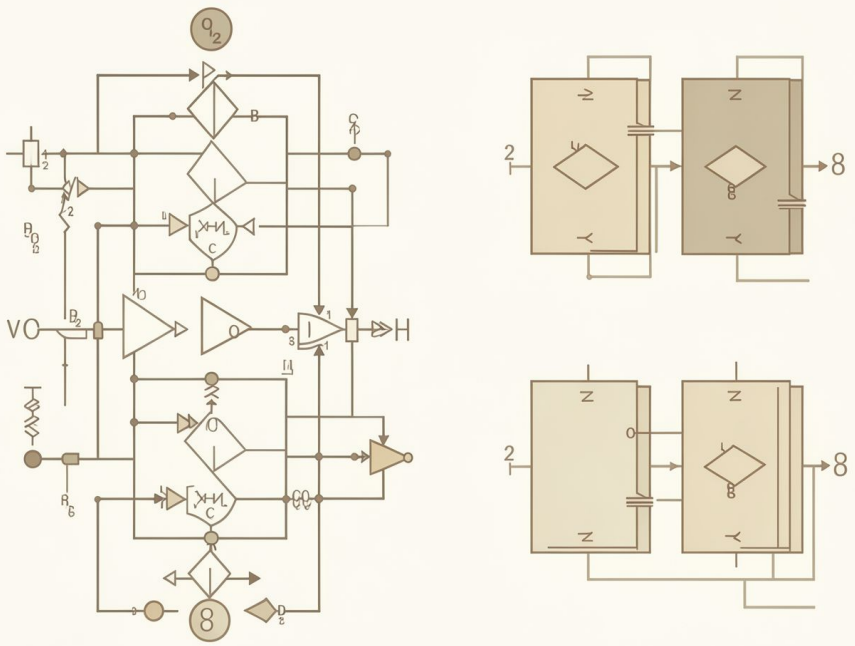
Directly reduces physical logic gate counts. Minimized formulas translate one-to-one into fewer transistors.

Hardware Payoff

Smaller chips, lower power consumption, and fewer manufacturing defects — all from fewer gates.

Before Minimization

Complex multi-gate circuit with redundant logic



After Minimization

Simplified minimal two-gate implementation

Made with GAMMA

The SAT Problem (Boolean Satisfiability)

SLIDE 40

The Boolean Satisfiability problem, or **SAT**, asks a simple question: given a logical formula, is there any assignment of truth values to its variables that makes the entire formula evaluate to **True**? If yes, the formula is **satisfiable**. If no, it is **unsatisfiable**. While this sounds simple, as the number of clauses and variables grows, finding that winning assignment becomes incredibly difficult.

SAT Definition

Deciding if a logical formula is satisfiable — the foundational decision problem of theoretical computer science.

The Assignment

Finding a True/False setting for all variables that evaluates the entire formula to True.

Formula Size

Search space grows exponentially as variable counts increase — 2^n possible assignments for n variables.

Example SAT Formula

(P ∨ Q ∨ R) ∧ (¬P ∨ ¬Q) ∧ (¬P ∨ ¬R) ∧ (¬R ∨ ¬Q)

Each clause is a disjunction. The full formula is a conjunction of clauses — this is **Conjunctive Normal Form (CNF)**, the standard input format for SAT solvers.

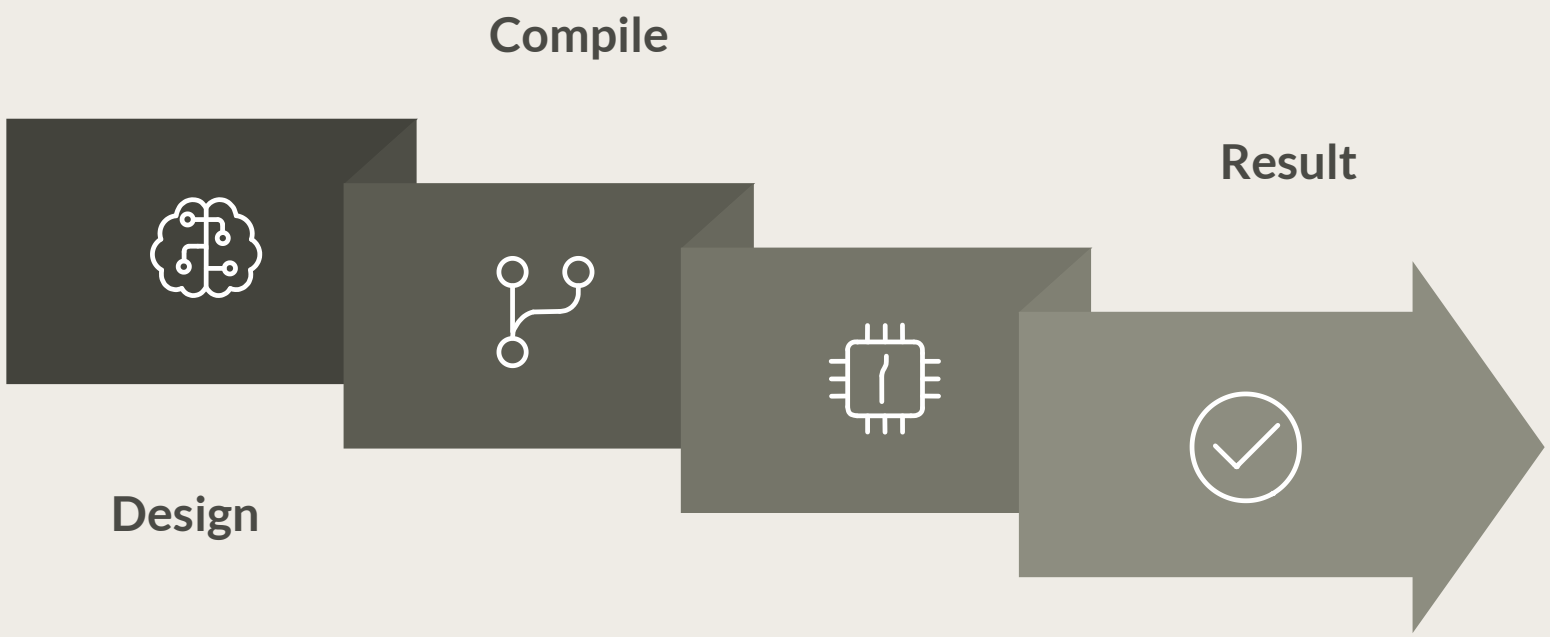
A SAT solver takes this formula as input and outputs either a satisfying assignment or a proof of unsatisfiability.

Made with GAMMA

SAT Solvers in Practice

SLIDE 41

While we are mathematically stuck on the theoretical worst-case complexity of SAT, computer scientists have made incredible practical breakthroughs. Modern software packages called **SAT solvers** can analyze formulas containing millions of variables with remarkable efficiency.



1

Industrial Scale

SAT solvers handle formulas with **millions of variables** — far beyond manual analysis.

2

Primary Arena

Digital circuit **verification engines** — the backbone of modern electronic design automation (EDA).

3

Complexity Trap

Hard to predict which formulas solve quickly. Runtime varies wildly across structurally similar inputs.

4

Unsatisfiability

Solvers are generally **far less effective** at proving unsatisfiability than finding satisfying assignments.

Made with GAMMA

P vs. NP and Cryptographic Chaos

SLIDE 42

The question of whether we can find a polynomial-time algorithm for SAT is known as the **P vs. NP problem** — arguably the most important open question in all of mathematics and computer science.

1

Class P

Problems solvable in **polynomial time**. Efficient algorithms exist. Examples: sorting, shortest path, primality testing.

2

Class NP

Nondeterministic polynomial-time search problems. Solutions are easy to *verify* once found, but hard to *find*.

3

SAT Status

SAT is **NP-Complete** — it represents the hardest class of NP problems. Every NP problem reduces to SAT.

The Encryption Risk

If someone proves **P = NP** by writing a fast SAT algorithm, the world changes — in two directions:

✓ Optimization Utopia

Scheduling, logistics, drug discovery, and AI optimization would become perfectly tractable.

⍰ Cryptographic Collapse

Standard public-key cryptography (RSA, ECC) would **break instantly**, rendering all online transactions insecure.

The standard assumption: **P ≠ NP**. No proof exists either way. The Clay Mathematics Institute offers a $1,000,000 prize for a solution.

Made with GAMMA

Pitfalls: Negation and Quantifier Swapping

SLIDE 43

As we transition to homework and exam preparation, let's isolate two major pitfalls that consistently cost students points. Both involve misapplying rules to quantified logical expressions.

Pitfall 1 — Negation of Quantifiers

⊗ **Common Error:** $\neg(\forall x: P(x)) \not\equiv \forall x: \neg P(x)$

When negating a quantified expression, students often negate the predicate but **forget to flip the quantifier**. The correct rule is to flip universal to existential:

> **Correct Rule:** $\neg(\forall x: P(x)) \equiv \exists x: \neg P(x)$

Example: If you negate "Everyone likes discrete structures," you do *not* get "No one likes discrete structures." You get: **"There is at least one person who does not like it."**

Pitfall 2 — Arbitrary Quantifier Swapping

⊗ **Common Error:** $\forall a\ \exists d: H(a, d) \not\equiv \exists d\ \forall a: H(a, d)$

Never swap universal and existential quantifiers arbitrarily. Swapping them **fundamentally changes the mathematical claim**.

> **$\forall a\ \exists d: H(a, d)$** — "For every a, there exists some d (possibly different for each a)."
> **$\exists d\ \forall a: H(a, d)$** — "There is one single d that works for all a simultaneously."

These are entirely different statements. The second is a far stronger claim than the first.

De Morgan's Law for Quantifiers

$\neg(\forall x: P(x)) \equiv \exists x: \neg P(x)$ | $\neg(\exists x: P(x)) \equiv \forall x: \neg P(x)$

Quantifier Order Rule

Order matters when quantifiers differ. Same-type quantifiers ($\forall\ \forall$ or $\exists\ \exists$) may be swapped freely; mixed types may not.

Made with GAMMA

Active Learning 1: Biconditional Truth Table

SLIDE 44

Pull out a sheet of paper or open your tablets. Populate this 4-row truth table for the conjunction of the implication and its converse. Fill in the columns for **P → Q**, **Q → P**, and then take their logical AND. Once finished, compare your final column to the biconditional definition. Take 90 seconds to write this out.

P	Q	P → Q	Q → P	(P → Q) ∧ (Q → P)	P ↔ Q
T	T	?	?	?	?
T	F	?	?	?	?
F	T	?	?	?	?
F	F	?	?	?	?

Goal

Fill the truth table for **(P → Q) ∧ (Q → P)** and verify the result matches **P ↔ Q**.

Check Points

- Identify all rows where the final output is **True**
- Confirm the final column is identical to the P ↔ Q column
- Recall: P → Q is False only when P is True and Q is False

Made with GAMMA

Active Learning 2: Algebraic Absorption Proof

SLIDE 45

Let's practice writing an algebraic logic proof. We want to prove the classic **Absorption Law**: A ∨ (A ∧ B) ≡ A. The four mathematical lines of the proof are written below — the justifications are blank. Write down the name of the propositional axiom or algebraic law that justifies each step. Think back to commutativity, distributivity, identity, and zero rules.

Step	Expression	Justification: _____
1	A ∨ (A ∧ B)	*Given — starting expression*
2	(A ∧ T) ∨ (A ∧ B)	Justification: _____
3	A ∧ (T ∨ B)	Justification: _____
4	A ∧ T	Justification: _____
5	A	Justification: _____

Step 1 → 2

Rewrite A as (A ∧ T) via **Identity Law**

Step 2 → 3

Factor out A via **Distributivity**: A ∧ (T ∨ B)

Step 3 → 4

Simplify T ∨ B = T via **Validity / Domination**

Step 4 → 5

Reduce A ∧ T = A via **Identity Law**

Made with GAMMA

Active Learning 3: Quantifier Translation

SLIDE 46

For our final active-learning check, let's translate **Goldbach's Conjecture** using only the domain of natural numbers ℕ. Our predicates are **E(n)**, meaning n is an even integer greater than 2, and **P(x)**, meaning x is prime.

Formal Translation Block

Statement: "Every even integer n > 2 is the sum of two primes."

Domain: All variables range over Natural numbers ℕ

Task 1: Write Goldbach's Conjecture in formal logic notation using quantifiers ∀, ∃ and predicates E(n), P(x).

$\forall n \in \mathbb{N}: E(n) \rightarrow \exists p\, \exists q \in \mathbb{N}: P(p) \land P(q) \land (n = p + q)$

Quantifier Negation Block

Task 2: Formulate the formal negation of the expression using De Morgan's Laws for Quantifiers.

Apply $\neg(\forall x: \varphi) \equiv \exists x: \neg\varphi$ and $\neg(\exists x: \varphi) \equiv \forall x: \neg\varphi$ step by step to negate the full expression.

Result: $\exists n \in \mathbb{N}: E(n) \land \forall p\, \forall q \in \mathbb{N}: \neg P(p) \lor \neg P(q) \lor (n \neq p + q)$

"There exists an even integer greater than 2 that is NOT the sum of any two primes."

Goldbach's Conjecture (1742) remains one of the oldest unsolved problems in number theory. It has been verified computationally for all even integers up to 4×10^{18}, but no general proof exists.

Made with GAMMA

Summary: Propositional Logic

SLIDE 47

We formalized the six key logical connectives using truth tables, established the completeness of our algebraic axioms, and proved that every logical expression can be written in standardized normal forms.

1. The 6 Connectives

Mastered **NOT, AND, OR, XOR, IFF, IMPLIES** — each defined by its truth table. These six connectives form a complete basis for propositional logic.

2. Implication Symmetry

Mathematical implication ignores causality. **P → Q matches its Contrapositive (¬Q → ¬P)** but does **NOT** match its Converse (Q → P) or Inverse (¬P → ¬Q).

3. Complete Axioms

A group of laws — commutativity, associativity, distributivity, identity, domination, double negation, De Morgan's — **algebraically proves logical equivalence** without truth tables.

4. Normal Forms (CNF/DNF)

Every formula reduces to an equivalent **Disjunctive Normal Form (DNF)** or **Conjunctive Normal Form (CNF)**. CNF is the standard input format for SAT solvers.

Connective	Symbol	Equivalent Form	Key Property
NOT	¬	—	Inverts truth value
AND	∧	—	True only if both true
OR	∨	—	False only if both false
IMPLIES	→	¬P ∨ Q	≡ Contrapositive; ≢ Converse
IFF	↔	(P→Q) ∧ (Q→P)	True when P and Q match
XOR	⊕	¬(P↔Q)	True when P and Q differ

Made with GAMMA

Summary: Predicate Logic and SAT Solving

A unified view of the two major themes from today's lecture — from quantifier logic to computational complexity.

Predicate Logic

- **Quantifiers**: Scope defined by Universal and Existential variables
- **Order Importance**: Swapping quantifiers alters the entire meaning
- **Counter-models**: Concrete method used to disprove invalid formulas
- **Negation**: De Morgan's Laws extended to quantifiers

SAT Complexity

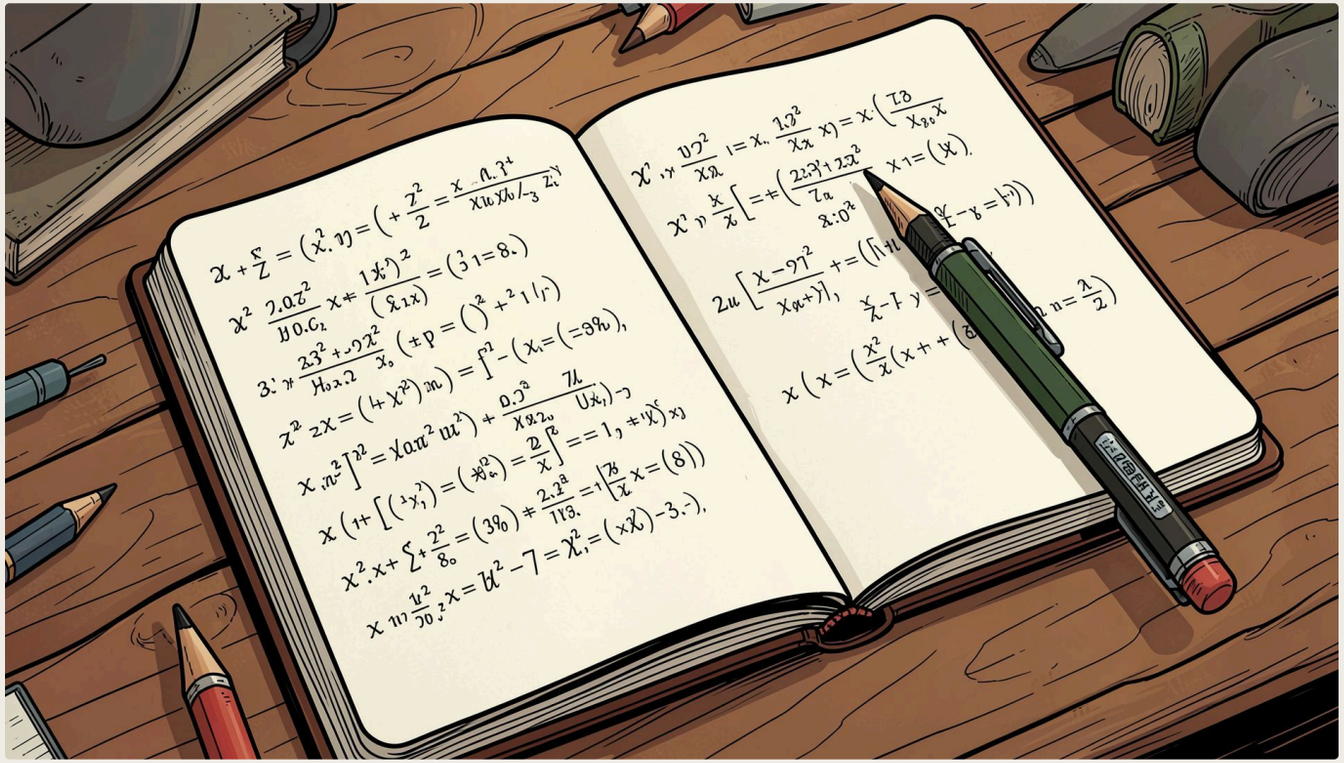
- **SAT Complexity**: Exponential growth bottleneck makes SAT hard
- **Satisfiability**: Determining if a formula can be made true
- **Practical Solvers**: Real-world tools that tackle SAT heuristically
- **P vs. NP**: The deep theoretical Millennium Problem at stake

In the second half of the lecture, we introduced universal and existential quantifiers, demonstrated how critical their ordering is, and explored the SAT problem alongside the theoretical significance of P vs. NP.

Made with GAMMA

Homework and Programming Lab 2

Two assignments are due next week — one pencil-and-paper proof exercise and one practical Java coding lab.



Pencil & Paper Assignments

- **Equivalence Proof**: Prove $(A \vee B) \wedge (A \vee \bar{B}) \equiv A$ via axioms
- **CNF Derivation**: Extract canonical CNF from a truth table

Java Programming Lab — COP 2800

- **Lab Task**: Refactor nested code conditionals into simplified DNF
- **Lab Objective**: Optimize conditional evaluation speed in COP 2800
- **Deliverable**: A Java class taking three boolean variables as parameters

Complete the reading before the lab session on Thursday. Office hours are posted on the syllabus.

Made with GAMMA



References and Further Reading

Primary Textbook

Mathematics for Computer Science (2018) — **Chapter 3: Logical Formulas**, Sections 3.3–3.5

Course Coordinates

COT 3100C — Miami Dade College. Syllabus available through the course repository.

Prerequisites

COP 2800 — Java Programming. Required background for the coding lab component.

Thank you for your active participation today. Complete Sections 3.3–3.5 before Thursday's lab session. See you in the computer lab!

Made with GAMMA